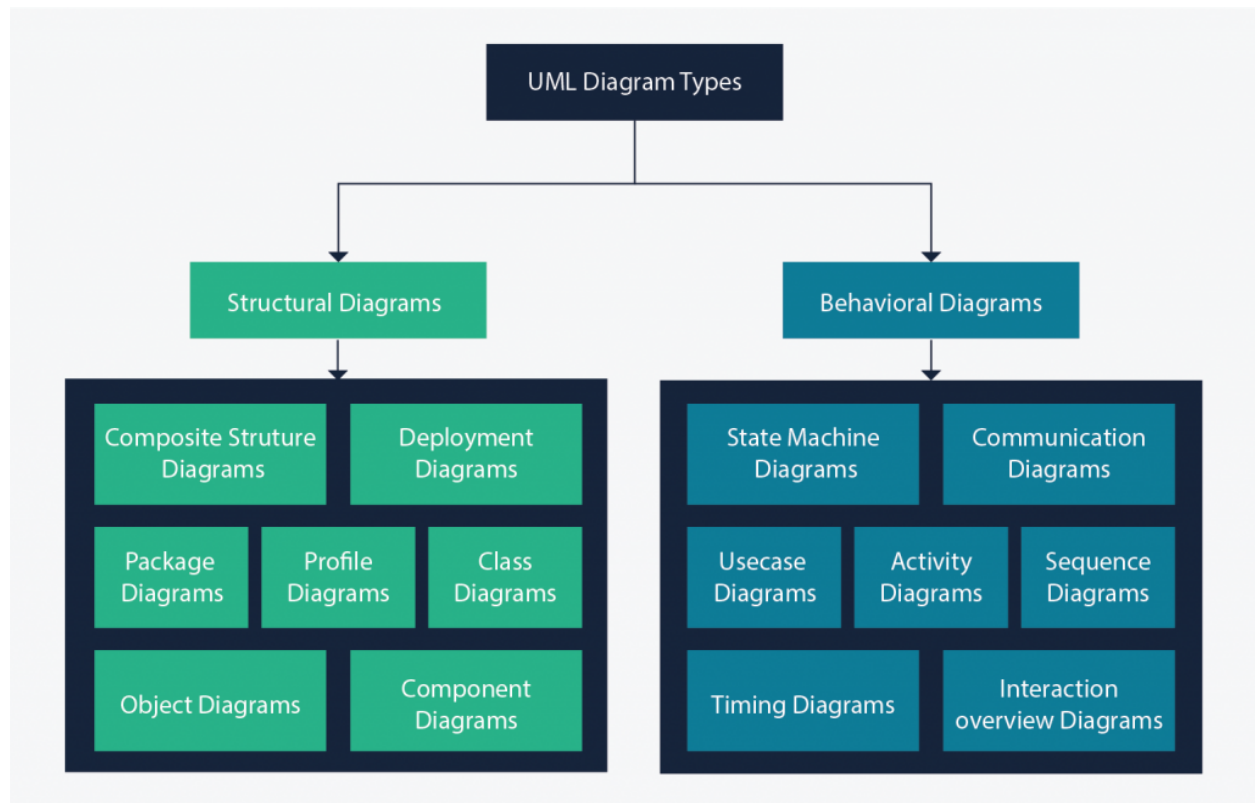


UML Diagram

UML stands for Unified Modeling Language

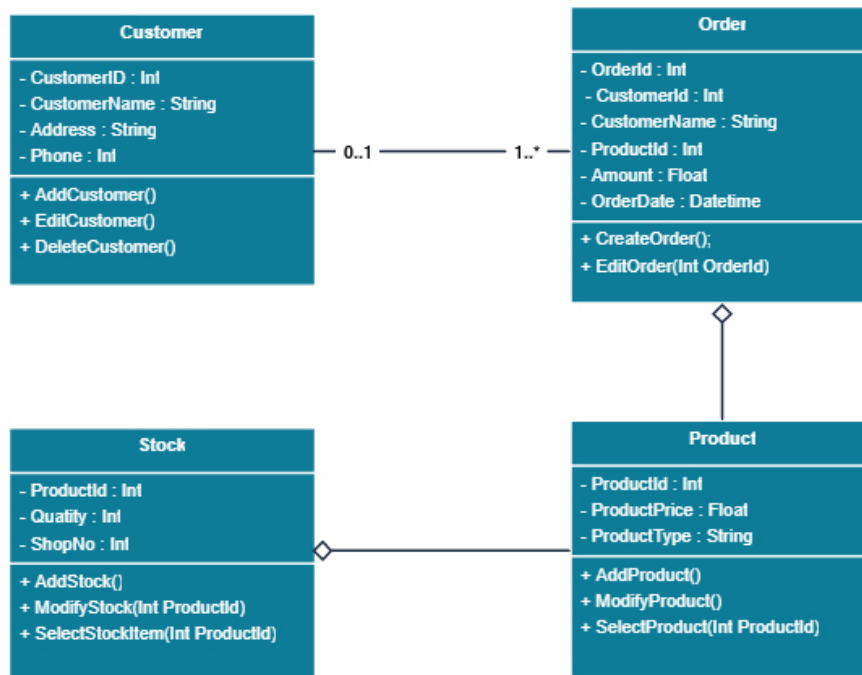


Structure diagrams show the things in the modeled system. In a more technical term, they show different objects in a system. **Behavioral diagrams** show what should happen in a system.

Class Diagram

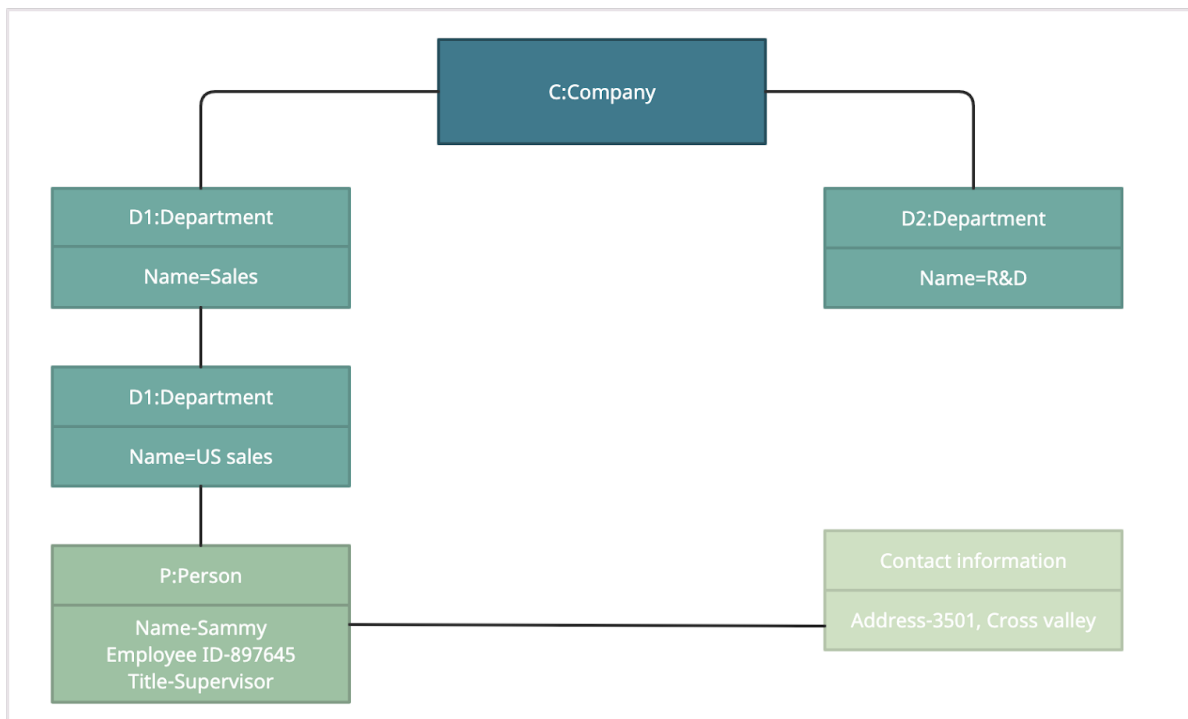
Class diagrams are the main building block of any object-oriented solution. It shows the classes in a system, attributes, and operations of each class and the relationship between each class.

Class Diagram for Order Processing System



Object Diagram

Object Diagrams, sometimes referred to as Instance diagrams are very similar to class diagrams. Like class diagrams, they also show the relationship between objects but they use **real-world examples**. They show what a system will look like at a given time. Because there is data available in the objects, they are used to explain complex relationships between objects.



Component Diagram

A component diagram displays the structural relationship of components of a software system. These are mostly used when working with complex systems with many components. Components communicate with each other using **interfaces**. The interfaces are linked using connectors.

Parts:

1. Component

Represent modular parts of the system that encapsulate functionalities.

2. Interfaces

Define how components communicate with each other, ensuring that components can be developed and maintained independently.

3. Relationships

Lines and arrows.

- Dependency (dashed arrow): Indicates that one component relies on another.
- Association (solid line): Shows a more permanent relationship between components.

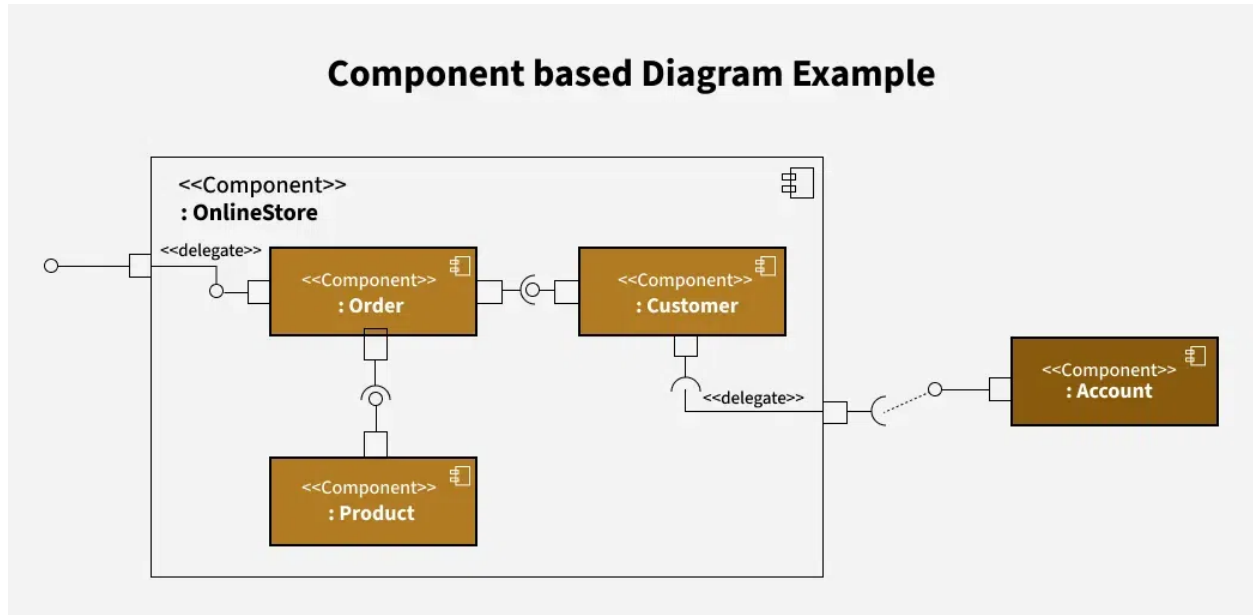
- Assembly connector: Connects a required interface of one component to a provided interface of another.

4. Ports

Allow for more precise specification of interaction points, facilitating detailed design and implementation.

5. Artifacts

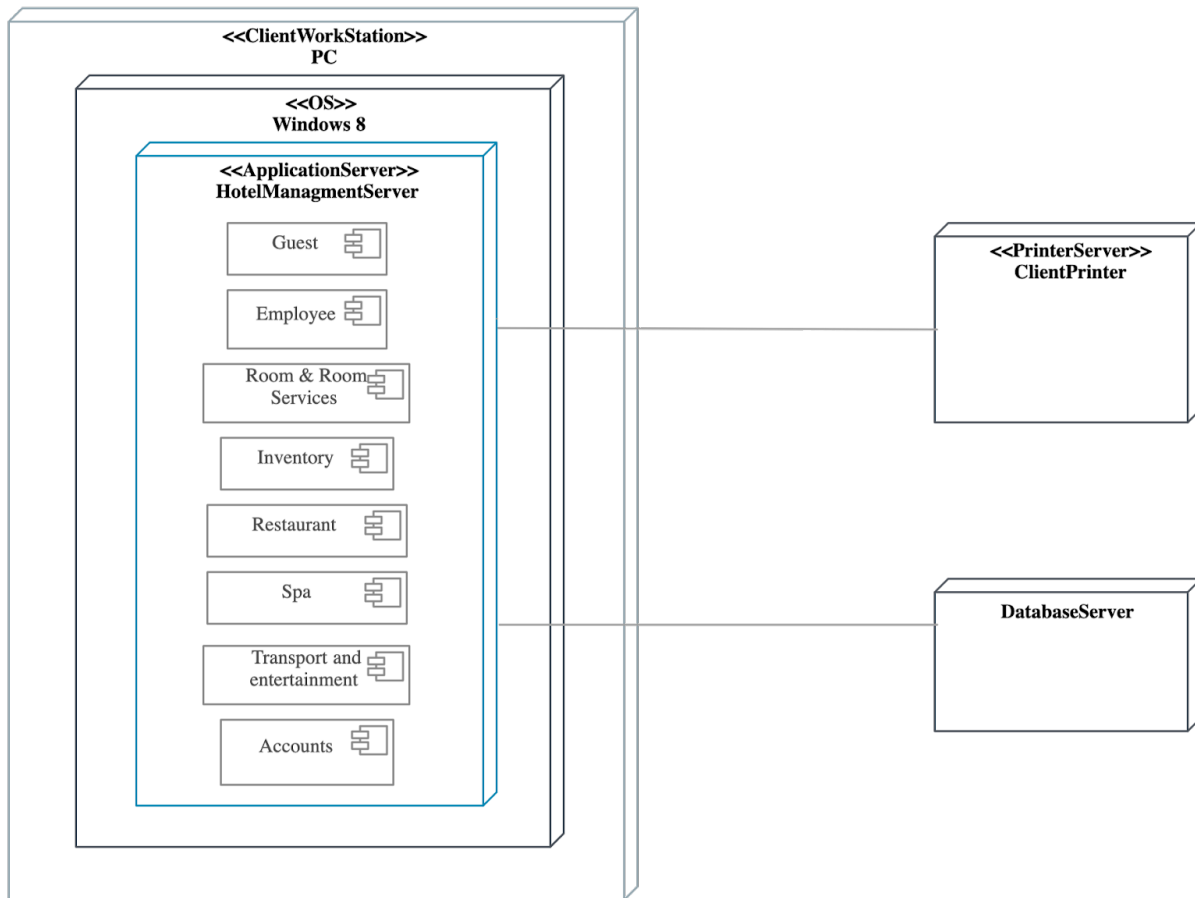
Show how software artifacts, like executables or data files, relate to the components.



Deployment Diagram

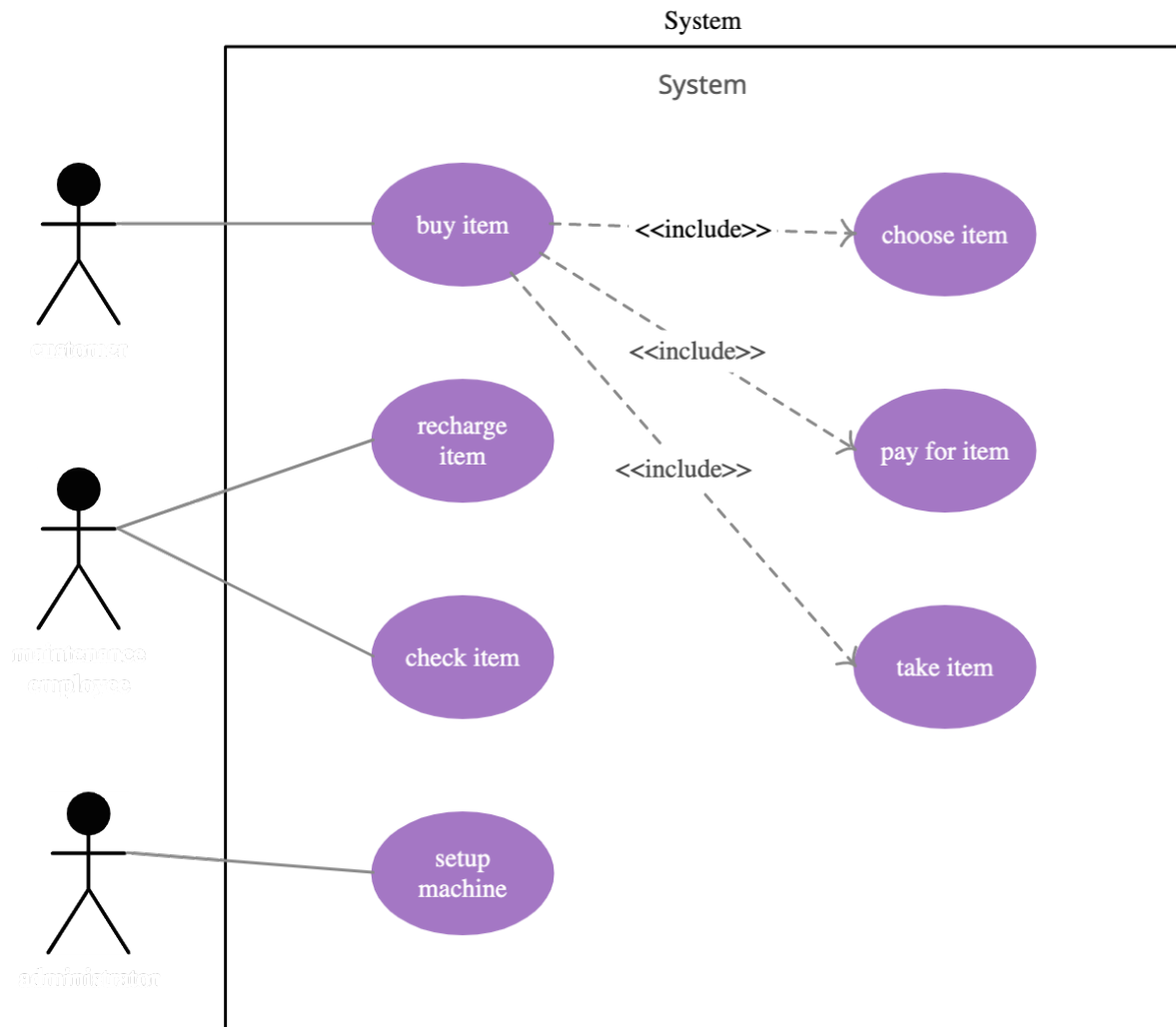
A deployment diagram shows the hardware of your system and the software in that hardware. Deployment diagrams are useful when your software solution is **deployed across multiple machines** with each having a unique configuration. Below is an example deployment diagram.

HOTEL MANAGEMENT SYSTEM



Use Case Diagram

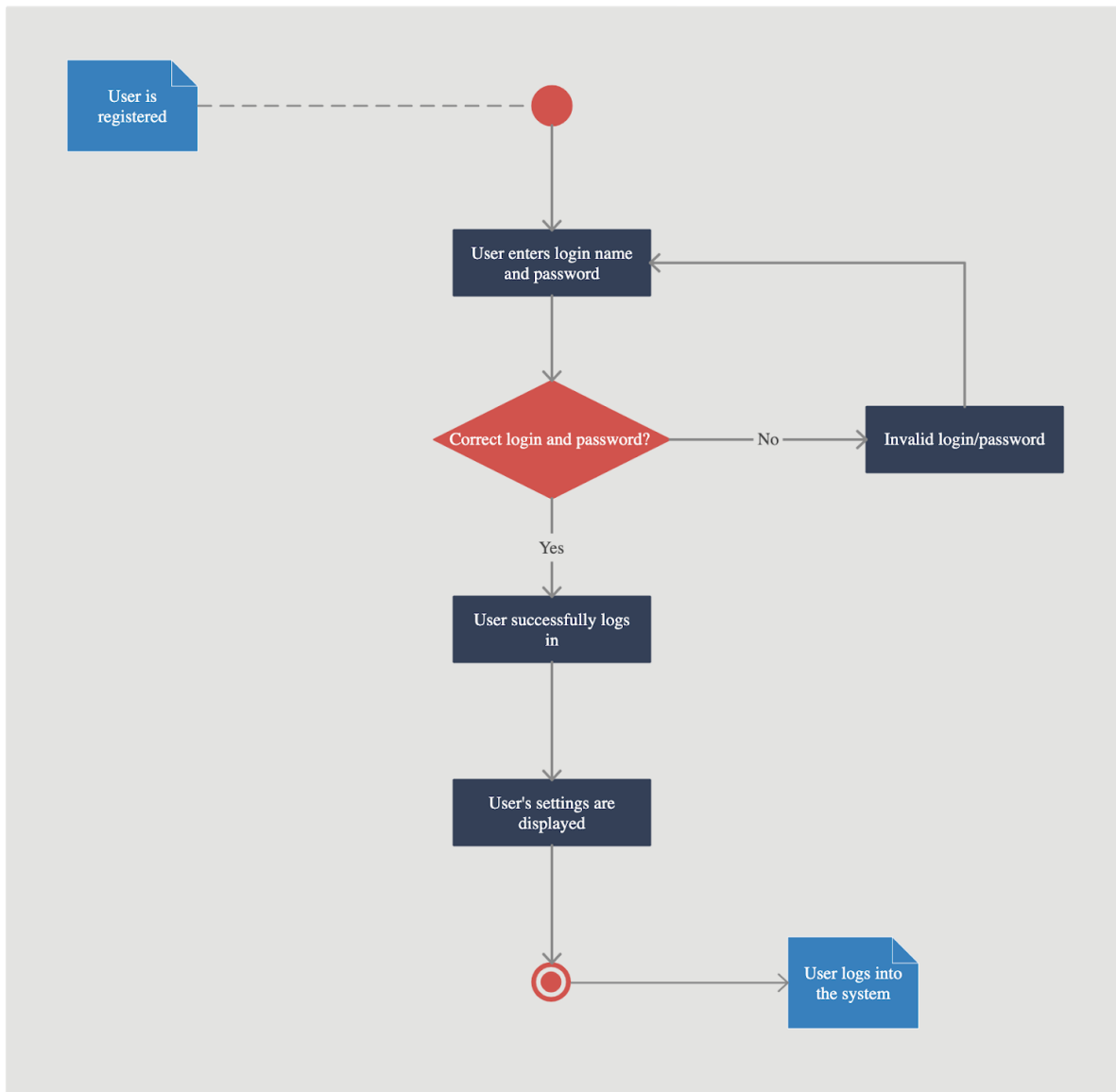
As the most known diagram type of the behavioral UML types, Use case diagrams give a graphic overview of the actors involved in a system, different functions needed by those actors and how these different functions interact.



Activity Diagram

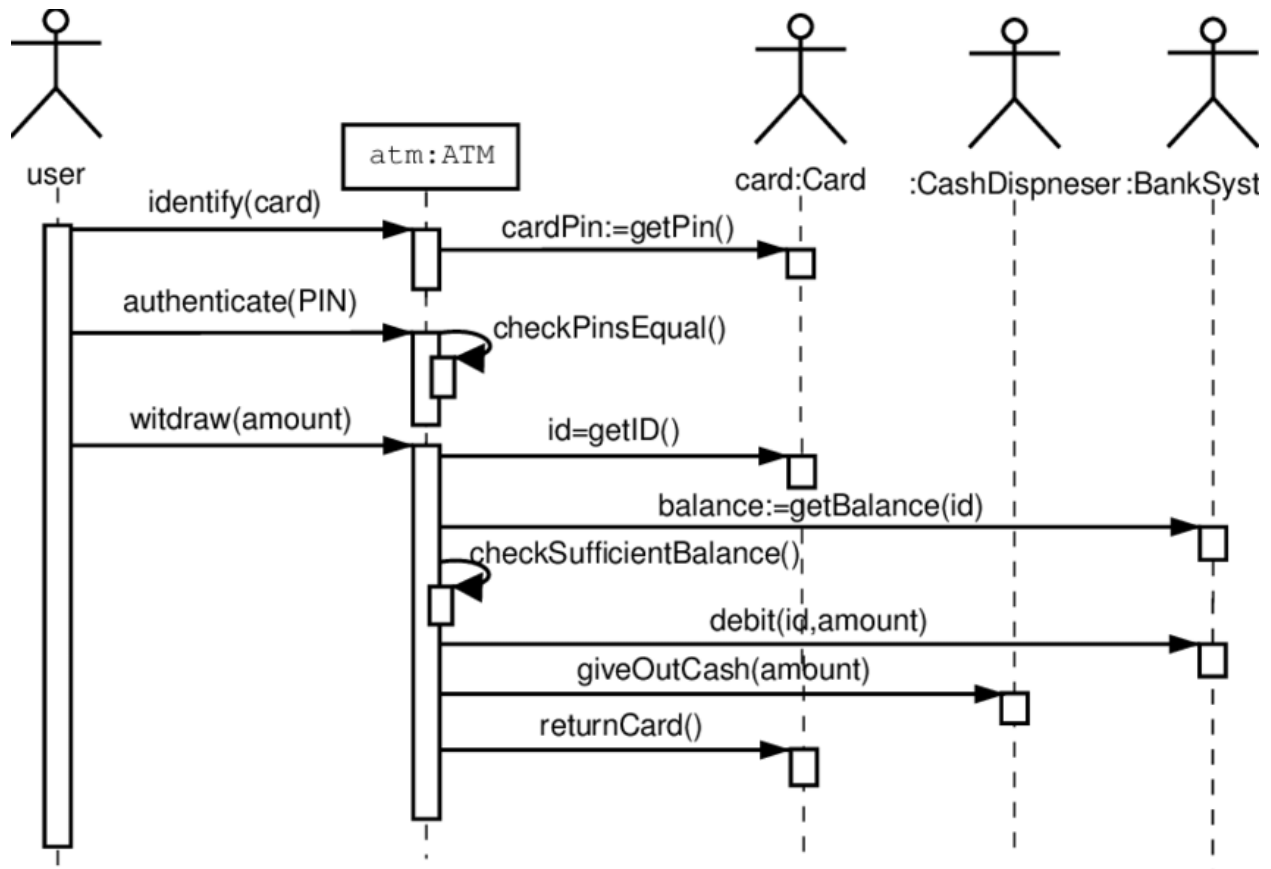
Activity diagrams represent workflows in a graphical way. They can be used to describe the business workflow or the operational workflow of any component in a system.

USER LOGIN SYSTEM for XYZ App



Sequence Diagram

Sequence diagrams in UML show how objects interact with each other and the order those interactions occur. **It's important to note that they show the interactions for a particular scenario.**



UML Diagram	Who Creates It	When It's Used	Why It's Useful (Purpose)
Class	Software architects, designers	Early design phase	To define the structure of classes, attributes, methods, and relationships
Object	Designers, developers	After class diagram; during runtime modeling	To show instances (objects) and how they relate at a specific moment
Component	Architects, dev leads	During high-level system design	To model physical components (e.g., modules, libraries, services) and dependencies
Deployment	DevOps, architects	Deployment planning	To show hardware and software environment , e.g., servers, containers, networks
Use Case	Business analysts, product owners	Early requirements phase	To capture functional requirements and interactions between users

			and the system
Activity	Analysts, developers	To model workflows or algorithmic logic	To show process flows , decisions, parallel actions (like a flowchart)
Sequence	Designers, developers	To specify behavior of use cases or methods	To model object interactions over time — who calls whom, in what order

Diagram	Requirement	Design	Implementation/D deployment
Class			
Object			
Component			
Deployment			
Use Case			
Activity			
Sequence			

Who Benefits?

- **Product Owners / Analysts** → Use case diagrams to define scope.
- **Architects** → Class, component, deployment diagrams for structure.
- **Developers** → Sequence, object, activity diagrams to guide implementation and logic.
- **DevOps** → Deployment diagrams to configure environments.